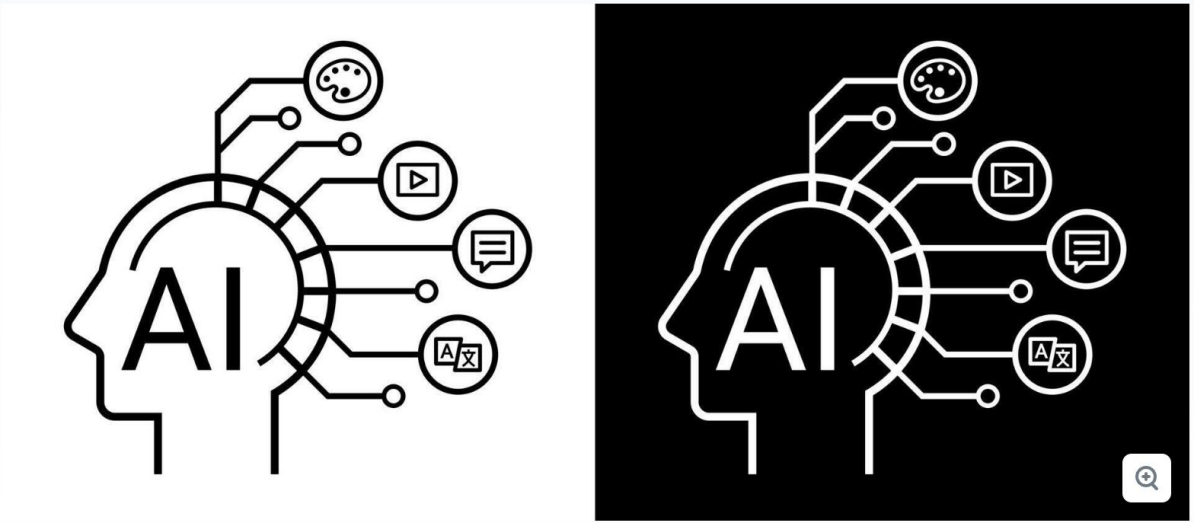
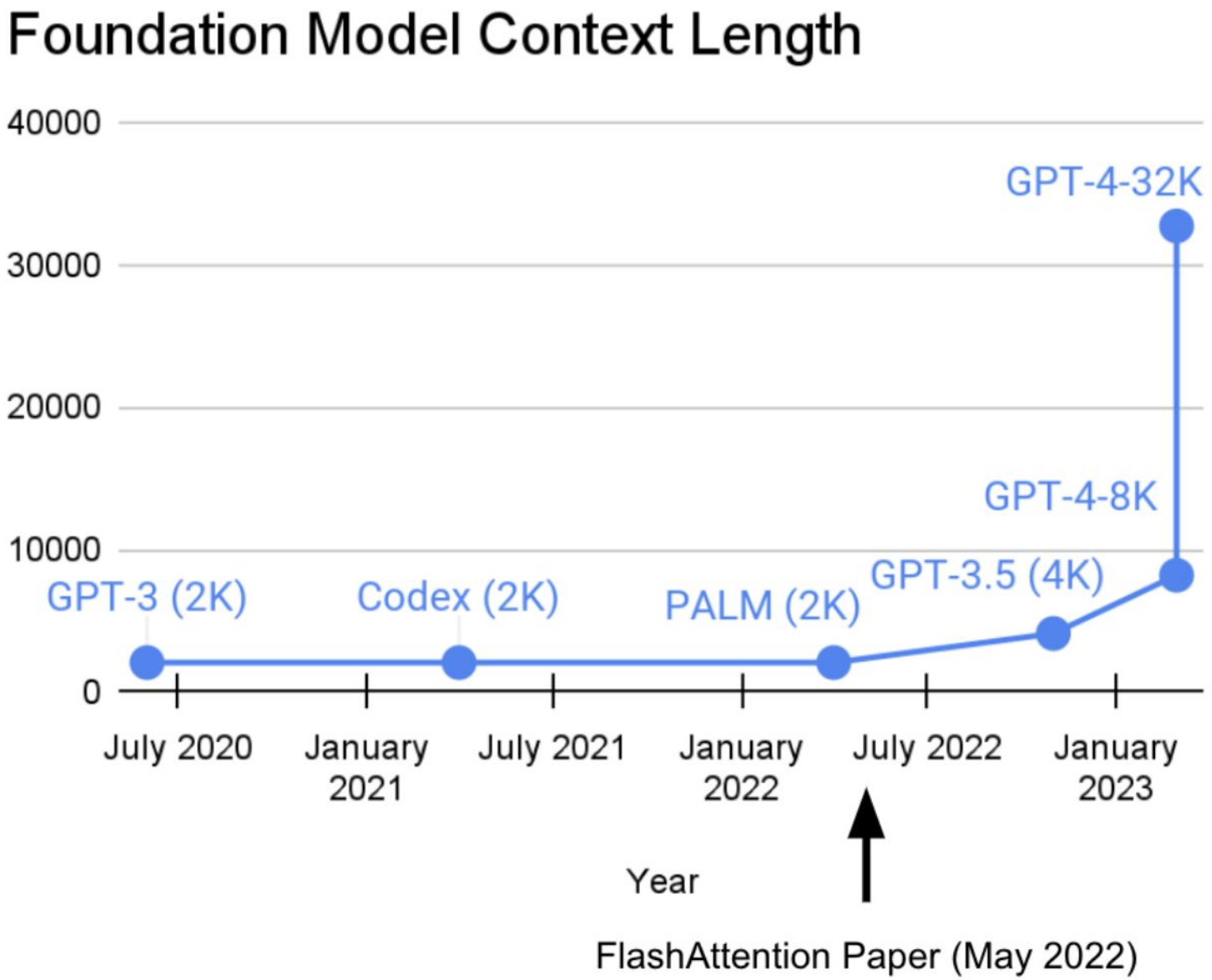
AIMS 5740
Generative Artificial Intelligence

(2026 Term 2)



Computer Science & Engineering
The Chinese University of Hong Kong

Context Length is Growing Rapidly



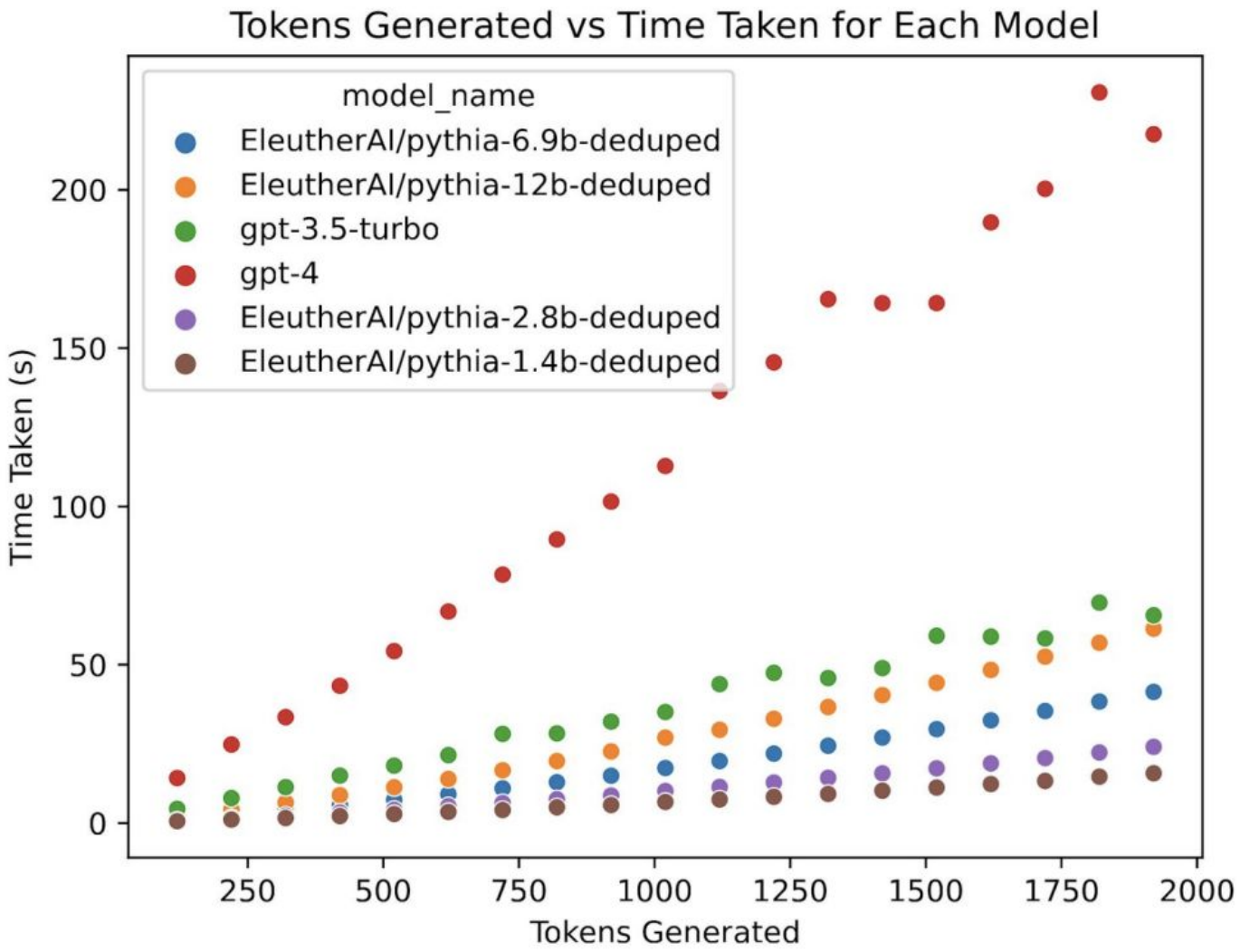
Issues with Transformers

- Training: quadratic time complexity
  - Expensive for long sequence modeling (e.g., video or DNA modeling)

- Inference: linear memory complexity
  - Requires storing KV cache for each token
  - High memory burden

Issues with Transformers

Mainly concerned with long text lengths



Modern Linear Recurrent Models

Use linear recurrence for parallel training

- Gated linear RNNs (HGRN, Griffin, ...)
- State-space models (S4, Mamba, ...)
- Linear attention (RetNet, GLA, xLSTM, DeltaNet, ...)

Modern Linear Recurrent Models

Use linear recurrence for parallel training

- Gated linear RNNs (HGRN, Griffin, ...)
- State-space models (S4, Mamba, ...)
- Linear attention (RetNet, GLA, xLSTM, DeltaNet, ...)

Mamba

Mamba: Linear-Time Sequence Modeling with Selective State Spaces

Albert Gu[*1] and Tri Dao[*2]

[1]Machine Learning Department, Carnegie Mellon University

[2]Department of Computer Science, Princeton University
agu@cs.cmu.edu, tri@tridao.me

Abstract

Foundation models, now powering most of the exciting applications in deep learning, are almost universally based on the Transformer architecture and its core attention module. Many subquadratic-time architectures such as linear attention.

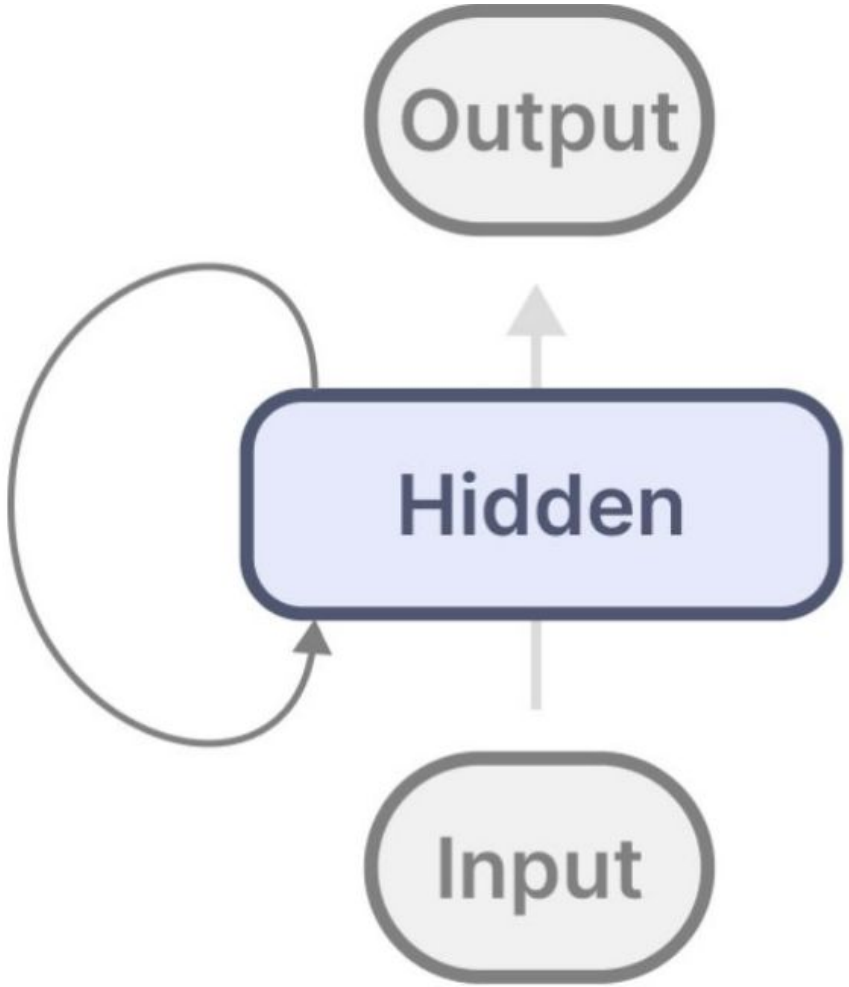
Revisiting RNNs

Compress information into a hidden state

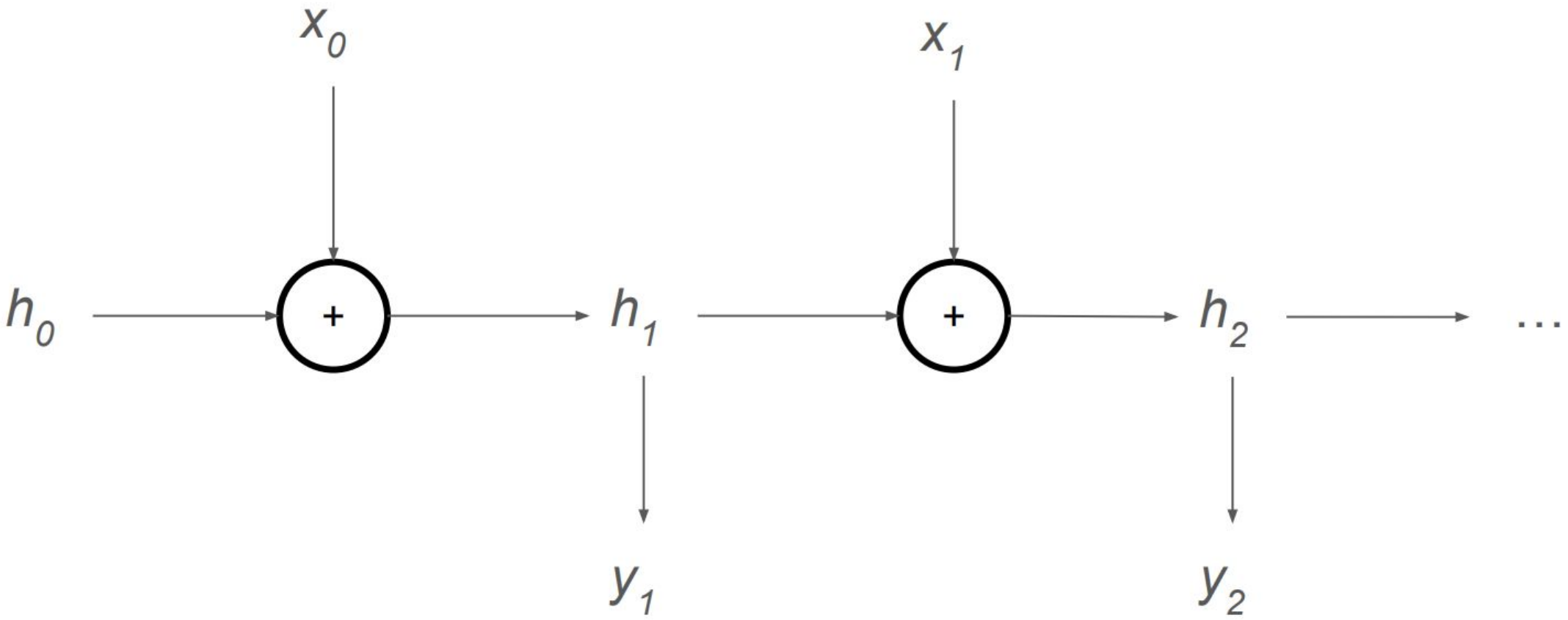
- only care about *h* and *x*

Questionable performance on very long data

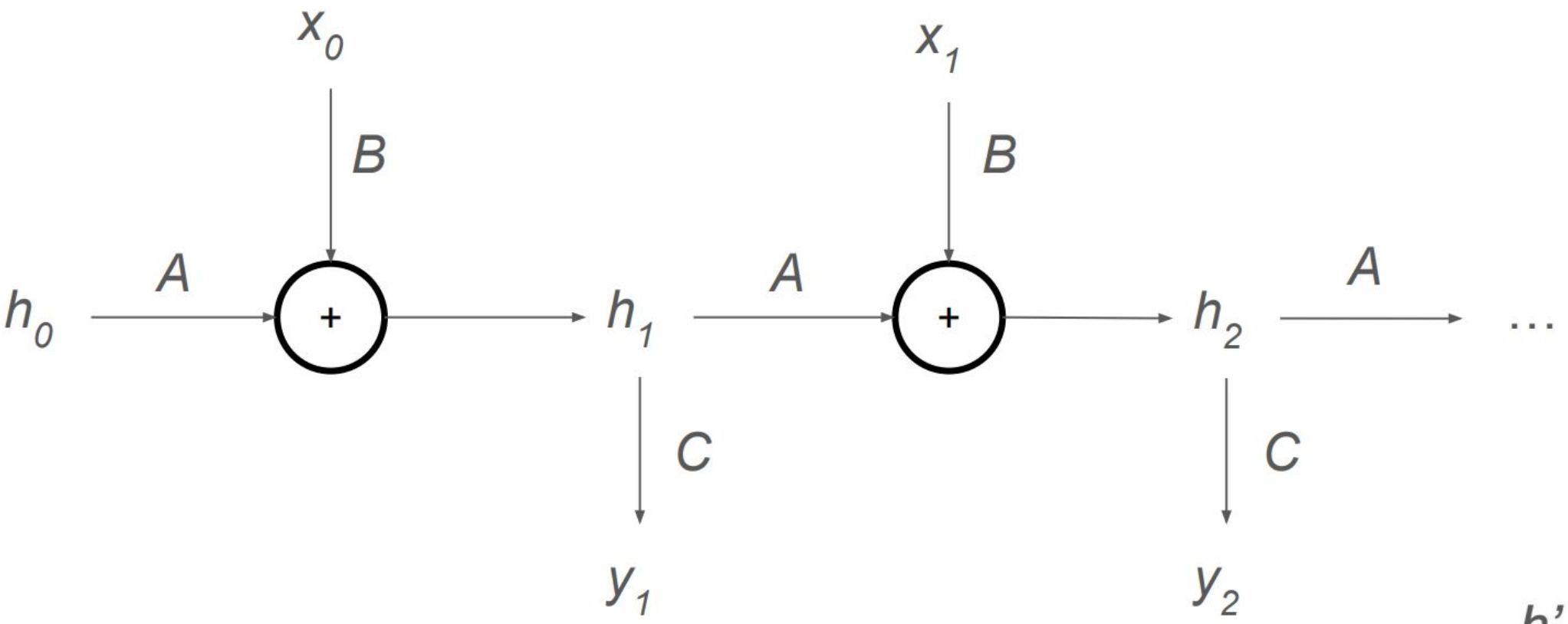
- can’t train in parallel



State Space Models



State Space Models



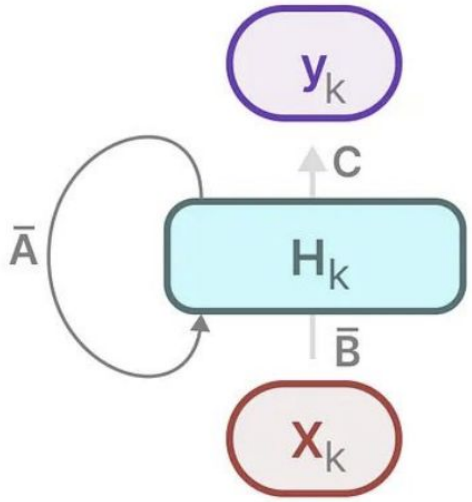
$h' = Ah + Bx$

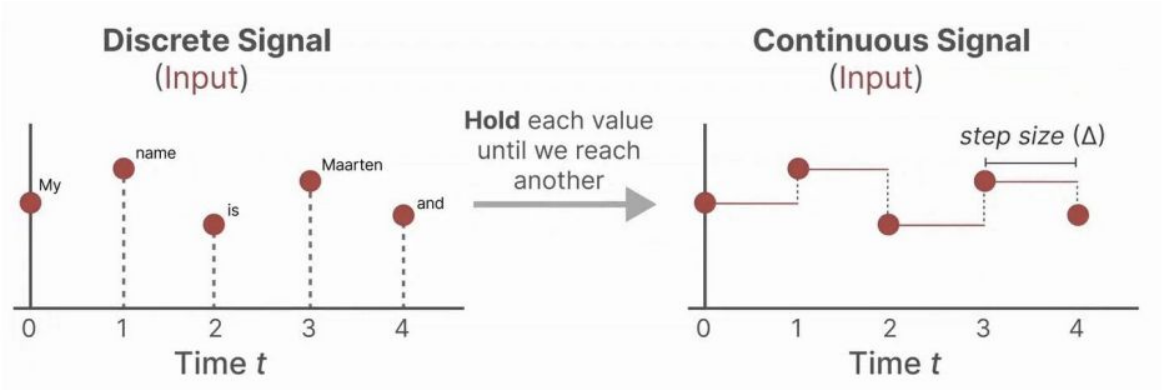
$y = Ch$

A: how much the state is preserved
B: how much new inputs affect the hidden state
C: converts the hidden state to an output

State Space Models

- Constant time inference
- Must convert discrete data to continuous using Δ
- Parallelizable via convolution due to *A, B, C* constant





$$\bar{A} = \exp(\Delta A)$$

$$\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$$

$$\bar{K} = (C\bar{B}, C\bar{A}\bar{B}, ..., C\bar{A}^{k}\bar{B}, ...)$$

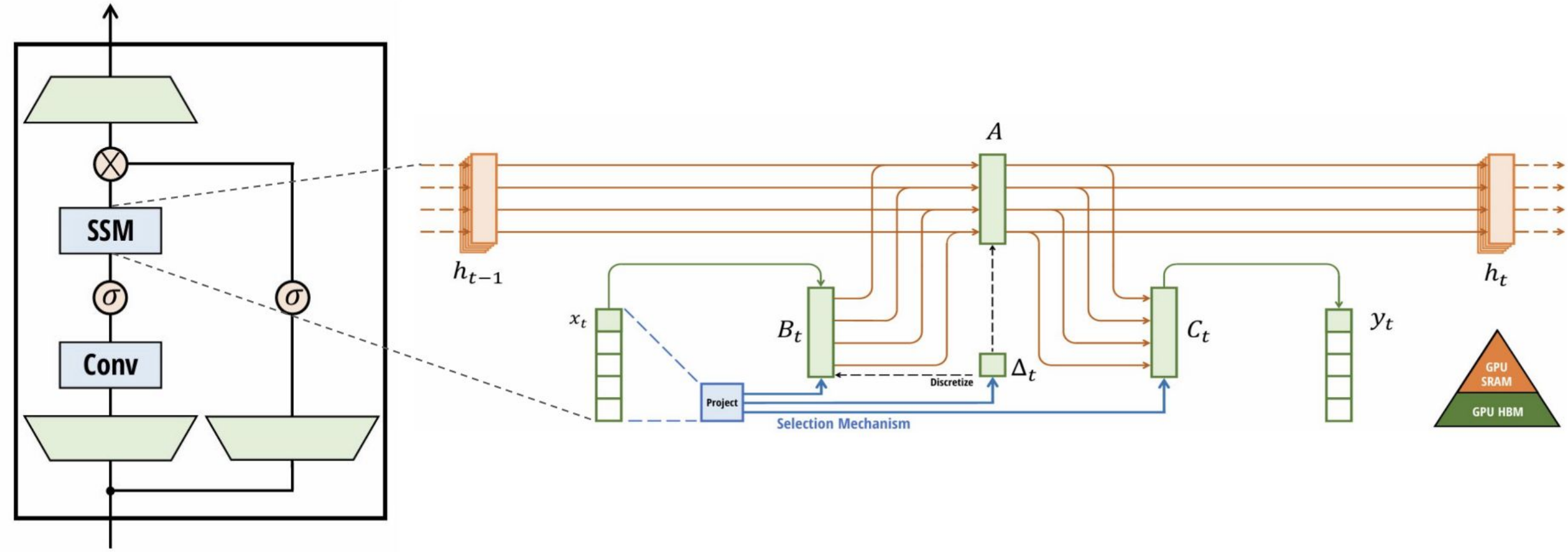
$$y = x * \bar{K}$$

Problems with SSM

- All tokens are compressed into the hidden state, mostly irreversibly
- Requires time invariance, so must treat all tokens equally in the computation of the next state
    - i.e. the parameters are constant

How can we restore the capabilities of transformers while keeping fast inference on long generations?

Mamba



Mamba block and at its core Selective SSM, or S6

Mamba: Key Idea

- “The fundamental problem of sequence modeling is *compressing context into a smaller state.*”
- **Selectivity:** the ability to **focus on or filter out** inputs into a sequential state
- A selection mechanism that controls how content affect hidden states(context): **attention in disguise!**

Mamba: Key Idea

- By letting parameters matrices that affect interactions along the sequence to be **input-dependent**!
  - Learn linear projectors sB,sC, sΔ to form the matrices from input.
  - Dimension L makes the model time varying.

Algorithm 1 SSM (S4)

Input: $x : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
Output: $y : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
1: $\boldsymbol{A} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $\boldsymbol{B} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
3: $\boldsymbol{C} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
4: $\Delta : (\mathsf{D}) \leftarrow \tau_\Delta(\text{Parameter})$
5: $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathsf{D}, \mathsf{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
6: $y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-invariant: recurrence or convolution
7: **return** y

Algorithm 2 SSM + Selection (S6)

Input: $x : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
Output: $y : (\mathsf{B}, \mathsf{L}, \mathsf{D})$
1: $\boldsymbol{A} : (\mathsf{D}, \mathsf{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
2: $\boldsymbol{B} : (\mathsf{B}, \mathsf{L}, \mathsf{N}) \leftarrow s_B(x)$
3: $\boldsymbol{C} : (\mathsf{B}, \mathsf{L}, \mathsf{N}) \leftarrow s_C(x)$
4: $\Delta : (\mathsf{B}, \mathsf{L}, \mathsf{D}) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
5: $\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathsf{B}, \mathsf{L}, \mathsf{D}, \mathsf{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
6: $y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-varying: recurrence (*scan*) only
7: **return** y

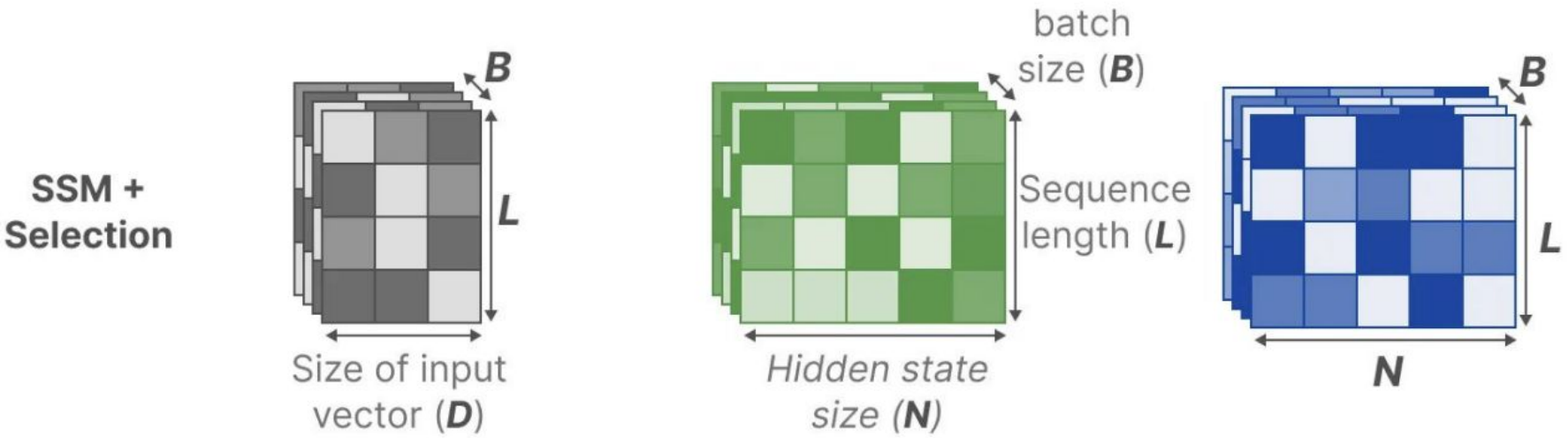
Mamba: Key Idea

- Now, for **every input token**, we have different Δ, B, C matrices.
- Together, they **selectively choose** how to modify the hidden state

Step size (Δ)
Resolution of the **input**
(discretization parameter)

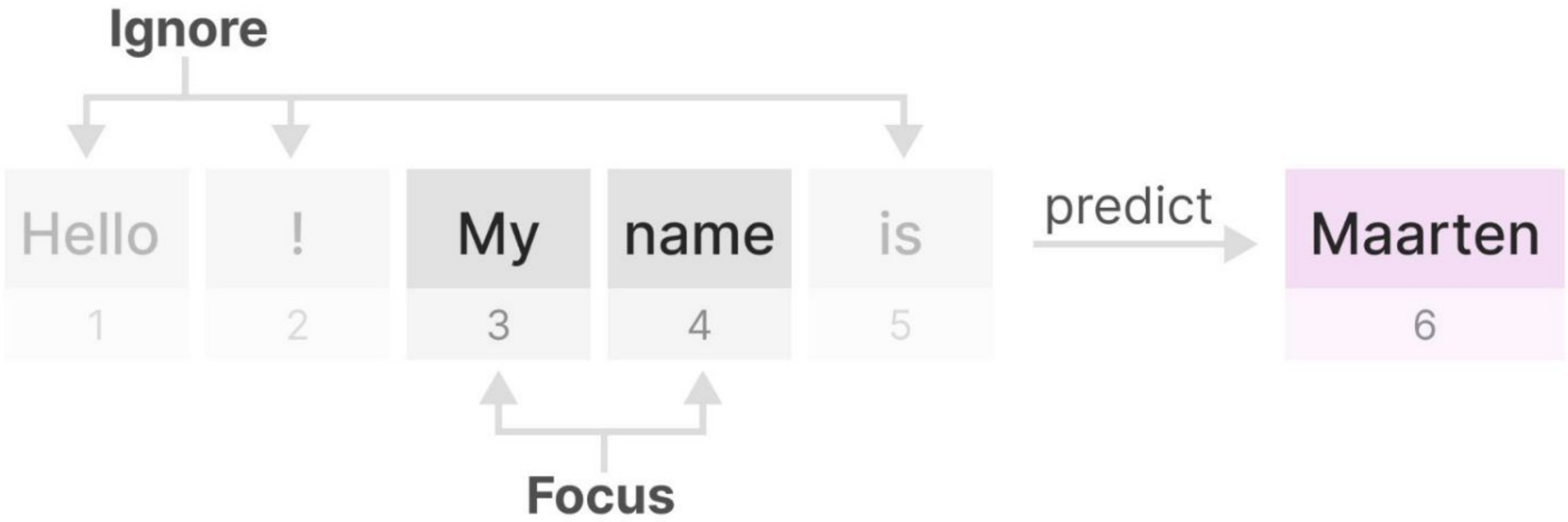
Matrix B
How the **input**
influences the state

Matrix C
How the **current state**
translates to the **output**



Selective State Space Models

- Example of the selective process:



How do we make this still fast?

When making the model input-dependent, we lose time invariance and the convolution analog.

- Naive recurrent mode uses ***O(BLDN)*** FLOPs. Seems hard to parallel.
- Prior SSMS use convolution ***(B,L,D)*** kernel bypasses the need to materialize ***(B,L,D,N)*** hidden states.

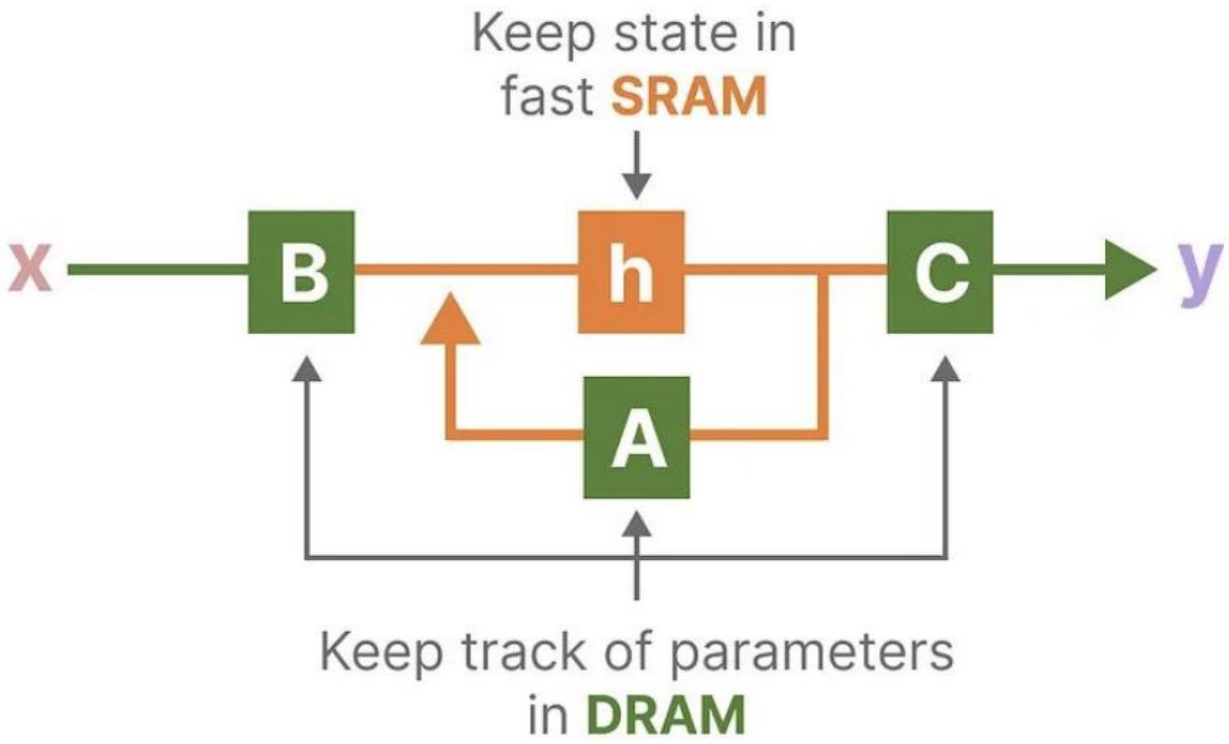
How to implement this selective process **without losing the computation efficiency**?

Key idea is that we utilize the properties of modern GPUs:

- Restore parallelizability using **parallel scan**
- Less memory IOs between SRAM and DRAM using **kernel fusion and recomputation**

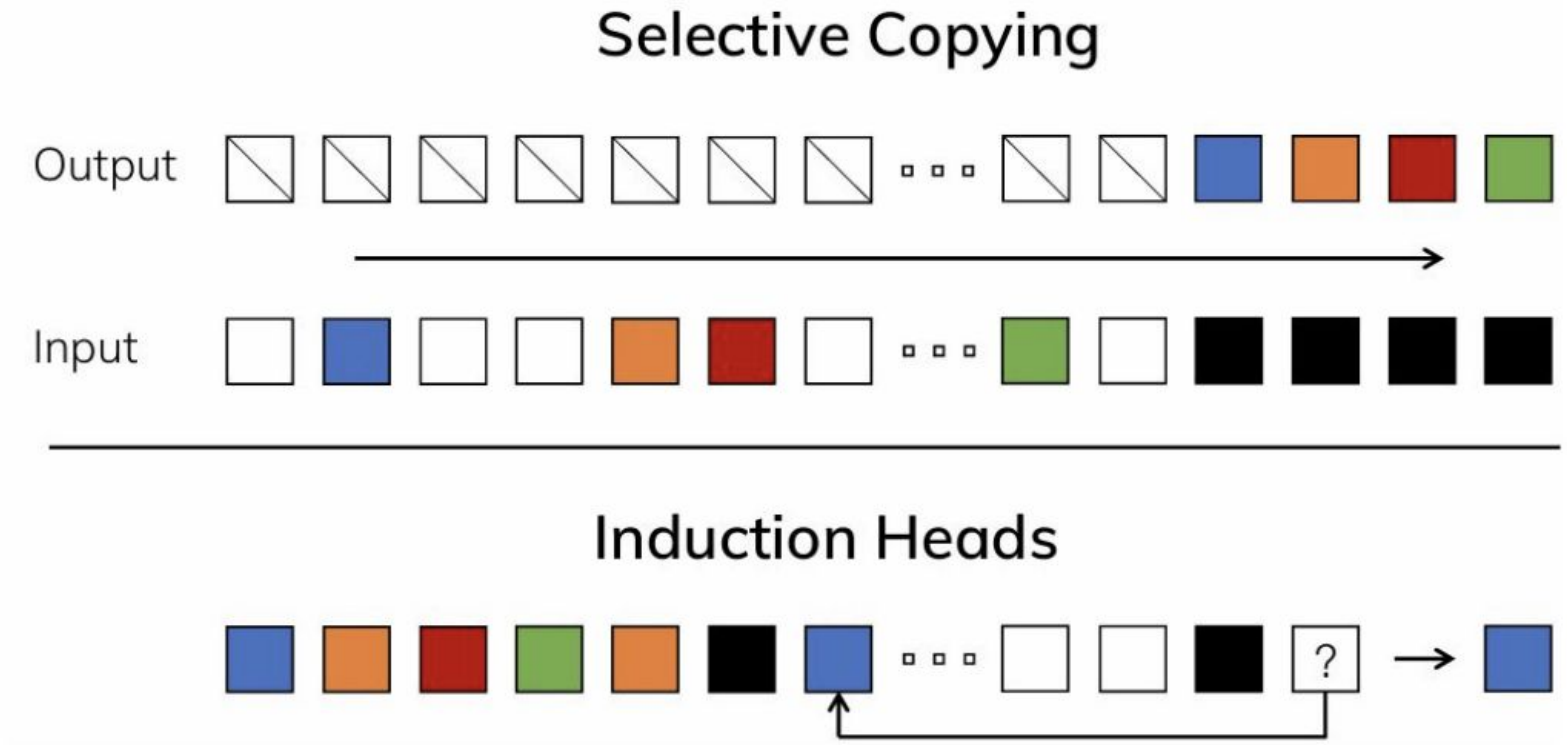
Hardware awareness algorithm

- Uses Classic Blelloch Scan to relieve sequential recurrence
- Reduces copying information between SRAM and DRAM through kernel fusion and recomputation, preventing writing intermediate results



Experiment

- Selective copying
- Induction heads
  - Difficult for state space models



Result

- Selection arch(modifying S4 to S6) easily solves selective copying
- Mamba solves induction heads and generalizes perfectly to million-length sequences

Model	Arch.	Layer	Acc.
S4	No gate	S4	18.3
-	No gate	S6	**97.0**
H3	H3	S4	57.0
Hyena	H3	Hyena	30.1
-	H3	S6	**99.7**
-	Mamba	S4	56.4
-	Mamba	Hyena	28.4
Mamba	Mamba	S6	**99.8**

Table 1: (**Selective Copying.**) Accuracy for combinations of architectures and inner sequence layers.

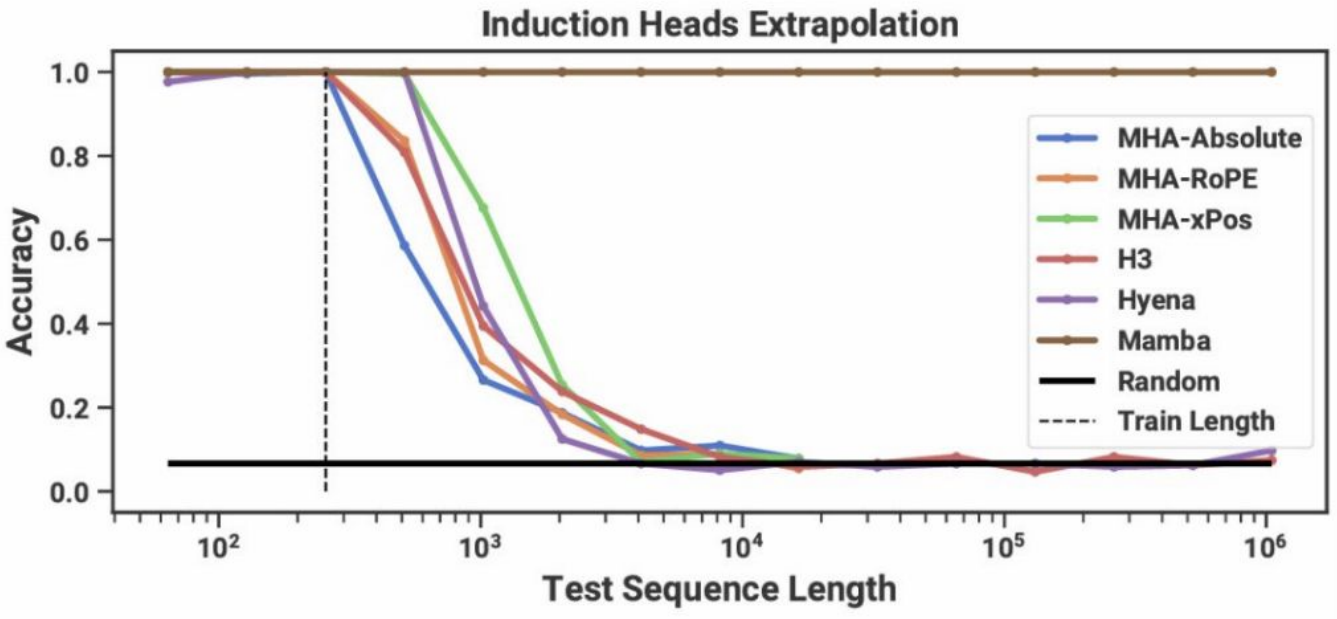
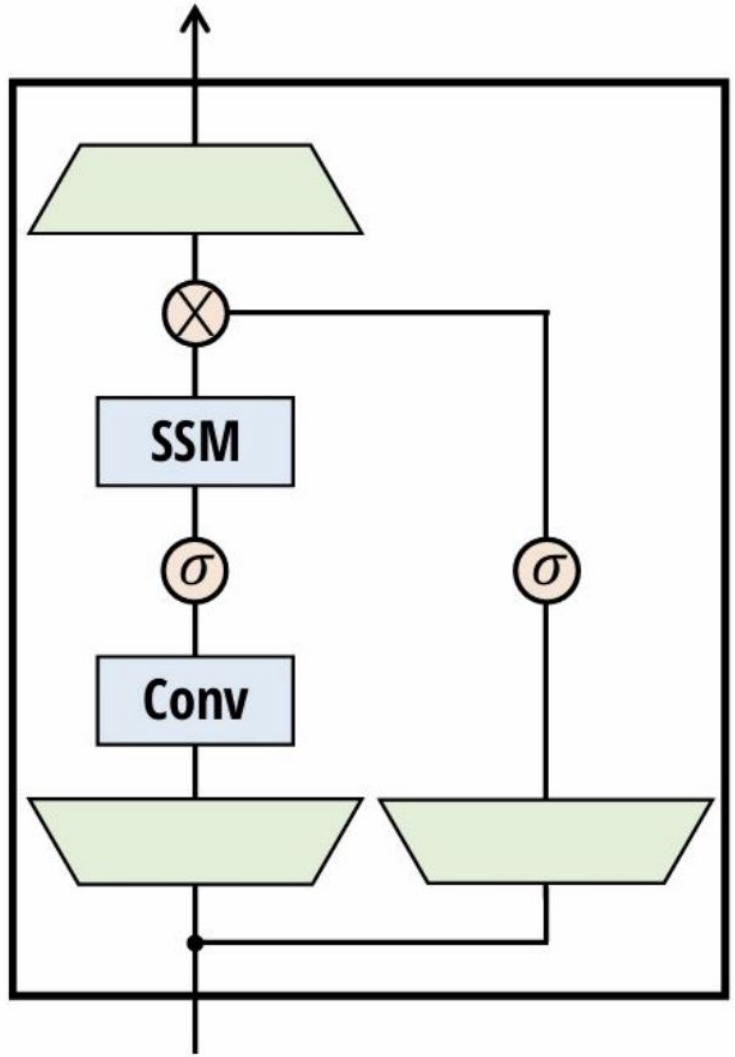


Table 2: (**Induction Heads.**) Models are trained on sequence length $2^8 = 256$, and tested on increasing sequence lengths of $2^6 = 64$ up to $2^{20} = 1048576$. Full numbers in Table 11.

Mamba: Summarization

- Mamba implements an S6 model (selection + structured state space model)
- Results in linear time inference scaling
- Relies on hardware optimization algorithm



Softmax Attention

Attention:

$$\text{Parallel training}: \quad \mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M})\mathbf{V} \quad \in \mathbb{R}^{L\times d}$$

$$\text{Iterative inference}: \quad \mathbf{o_t} = \sum_{j=1}^{t} \frac{\exp(\mathbf{q}_t^\top \mathbf{k}_j)}{\sum_{l=1}^{t} \exp(\mathbf{q}_t^\top \mathbf{k}_l)} \mathbf{v}_j \quad \in \mathbb{R}^{d}$$

where $\mathbf{M} \in \mathbb{R}^{L\times L}$ is the casual mask:

$$\mathbf{M}_{i,j} = \begin{cases} -\infty & \text{if } j > i \\ 1 & \text{if } j \leq i \end{cases}$$

Linear Attention = Standard Attention - Softmax

Linear attention (Katharopoulos et al. 2020):

$$\text{Parallel training}: \quad \mathbf{O} = \cancel{\text{softmax}}(\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M})\mathbf{V} \quad \in \mathbb{R}^{L\times d}$$

$$\text{Iterative inference}: \quad \mathbf{o_t} = \sum_{j=1}^{t} \frac{\cancel{\exp}(\mathbf{q}_t^\top \mathbf{k}_j)}{\cancel{\sum_{l=1}^{t} \exp(\mathbf{q}_t^\top \mathbf{k}_l)}} \mathbf{v}_j \quad \in \mathbb{R}^{d}$$

where $\mathbf{M}$ is the causal mask for linear attention:

$$\mathbf{M}_{i,j} = \begin{cases} 0 & \text{if } j > i \\ 1 & \text{if } j \leq i \end{cases}$$

Equivalent View: Matrix-Valued Hidden States

$$\mathbf{o}_t = \sum_{j=1}^{t} (\mathbf{q}_t^\top \mathbf{k}_j)\mathbf{v}_j$$

$$= \sum_{j=1}^{t} \mathbf{v}_j(\mathbf{k}_j^\top \mathbf{q}_t) \quad \mathbf{k}_j^\top \mathbf{q}_t = \mathbf{q}_t^\top \mathbf{k}_j \in \mathbb{R}$$

$$= \underbrace{\left(\sum_{j=1}^{t} \mathbf{v}_j \mathbf{k}_j^\top\right)}_{\mathbf{S}_t \in \mathbb{R}^{d\times d}} \mathbf{q}_t \qquad \text{By associativity}$$

Linear Attention = Linear RNN + Matrix-valued Hidden States

Let $\mathbf{S}_t = \sum_{j=1}^{t} \mathbf{v}_j \mathbf{k}_j^\top \in \mathbb{R}^{d \times d}$ be the matrix-valued hidden state, then:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top \quad \in \mathbb{R}^{d \times d}$$

$$\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \quad \in \mathbb{R}^{d}$$

- Linear attention implements **elementwise linear recurrence**.
- Linear attention has a **matrix-valued hidden state**, significantly increasing the state size.

Challenges in Training: the Parallel Form

$$\mathbf{O} = (\mathbf{Q}\mathbf{K}^\top \odot \mathbf{M})\mathbf{V} \in \mathbb{R}^{L \times d}$$

The time complexity is still quadratic in sequence length, which is problematic for long sequences.

Challenges in Training: the Recurrent Form

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top \quad \in \mathbb{R}^{d \times d}$$
$$\mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t \quad \in \mathbb{R}^{d}$$

Poor GPU utilization due to:

- Sequential computation limits parallelization opportunities across the sequence dimension.
- Rank-1 outer product updates and matrix-vector multiplications are not optimized for GPU tensor cores, which are designed for dense matrix-multiply operations (typically at least 16x16 matrix sizes).

Chunkwise Parallel Form

Chunkwise Form:

- Interpolates between recurrent and parallel forms.
- Splits a sequence of length L into L/C chunks of size C:
  - When $C = 1$, it reduces to the recurrent form.
  - When $C = L$, it reduces to the parallel form.
- **Key Property:** Chunkwise form is **NOT an approximation**—it computes the exact same output as the original formulation.

Chunkwise Parallel Form

Chunkwise form computes only the **last hidden state** per chunk.
Output is derived from:

- **Recurrent Form:** Historical context across chunks.
- **Parallel Form:** Local context within a chunk.

Notations

$$\mathbf{S}_{[i]} := \mathbf{S}_{iC} \in \mathbb{R}^{d \times d} \qquad \text{the last hidden state of chunk } i,$$

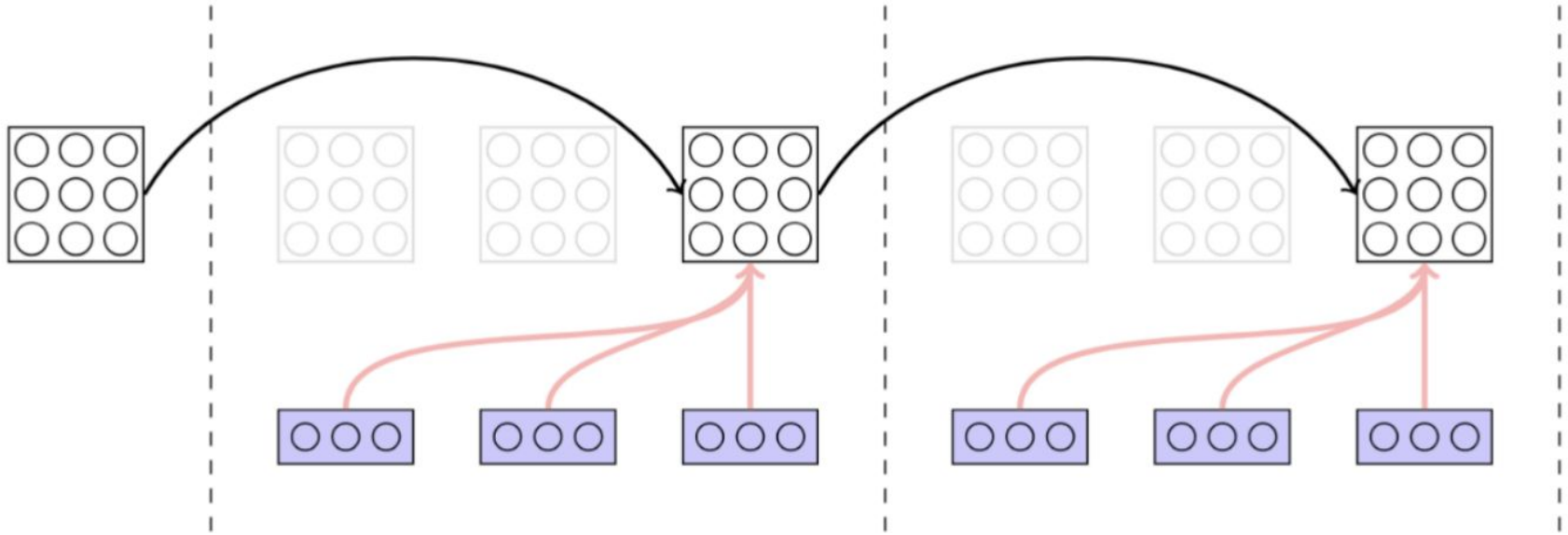
$$\mathbf{Q}_{[i]} = \mathbf{Q}_{iC+1:(i+1)C} \in \mathbb{R}^{C \times d} \qquad \text{the query block of chunk } i.$$

We define $\mathbf{K}_{[i]}, \mathbf{V}_{[i]}, \mathbf{O}_{[i]}$ in a similar way.

Chunkwise Parallel Form: Hidden State Update

Sequential Chunk-Level State Passing:

$$\boldsymbol{S}_{[i+1]} = \boldsymbol{S}_{[i]} + \boldsymbol{V}_{[i]}^{\top}\boldsymbol{K}_{[i]}$$

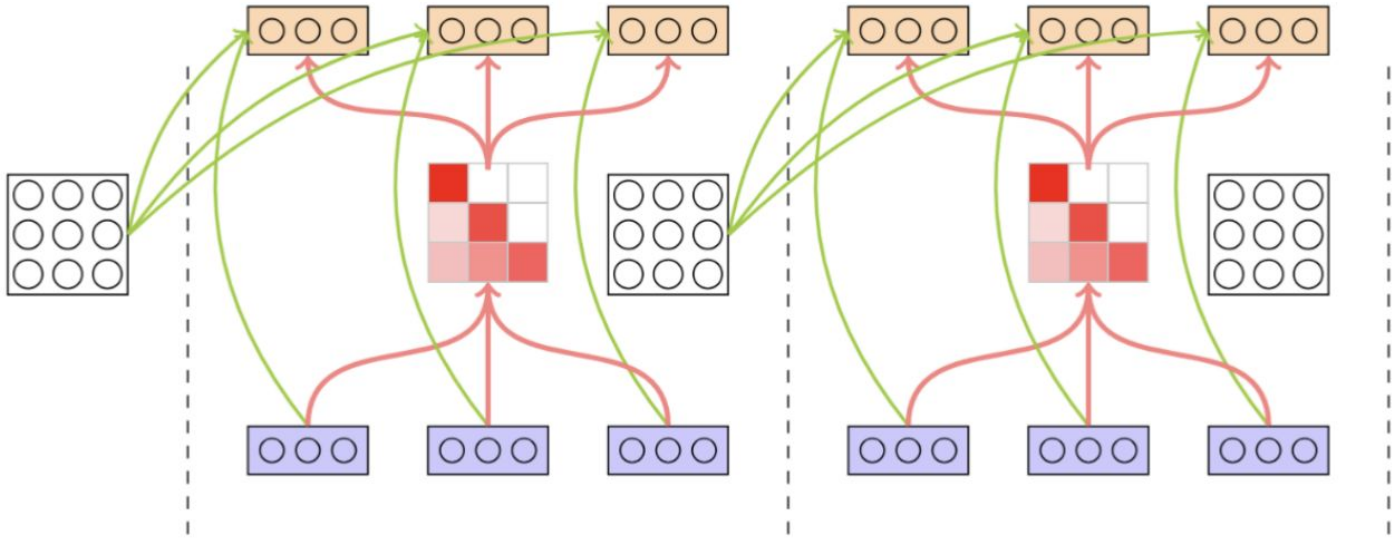


$$\mathbf{S}_{[t+1]} = \underbrace{\mathbf{S}_{[t]}}_{\mathbb{R}^{d\times d}} + \underbrace{\mathbf{V}_{[t]}^{\top}}_{\mathbb{R}^{d\times C}}\underbrace{\mathbf{K}_{[t]}}_{\mathbb{R}^{C\times d}} \quad \in \mathbb{R}^{d\times d}$$

Chunkwise Parallel Form: Hidden State Update

Parallel Output Computation:

$$\mathbf{O}_{[i]} = \mathbf{Q}_{[i]}\mathbf{S}_{[i]}^{\top} + \left(\mathbf{Q}_{[i]}\mathbf{K}_{[i]}^{\top} \odot \mathbf{M}\right)\mathbf{V}_{[i]}$$



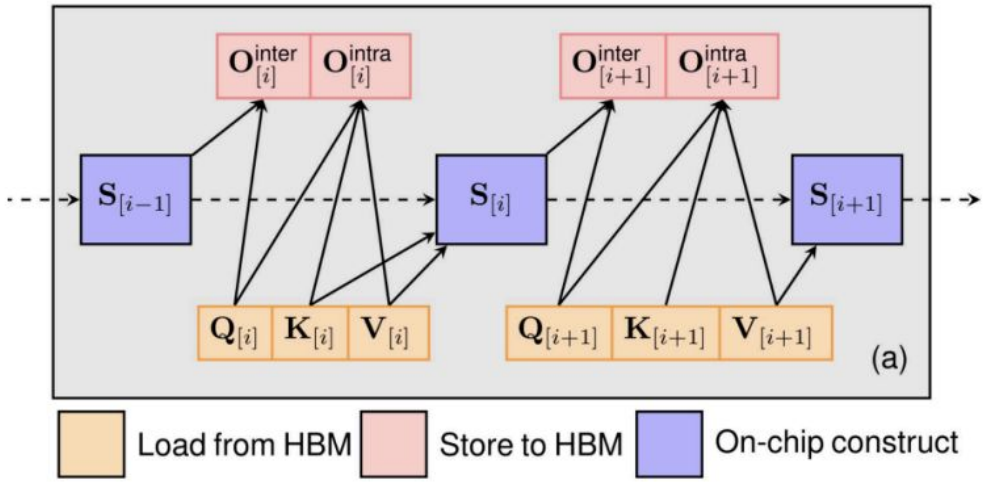
$$\mathbf{O}_{[t]} = \underbrace{\overbrace{\mathbf{Q}_{[t]}}^{\mathbb{R}^{C\times d}}\overbrace{\mathbf{S}_{[t]}^{\top}}^{\mathbb{R}^{d\times d}}}_{\text{inter-chunk: } \mathbf{O}_{[t]}^{\text{inter}}} + \underbrace{\overbrace{(\mathbf{Q}_{[t]}\mathbf{K}_{[t]}^{\top} \odot \mathbf{M})}^{\mathbb{R}^{C\times C}}\overbrace{\mathbf{V}_{[t]}}^{\mathbb{R}^{C\times d}}}_{\text{intra-chunk: } \mathbf{O}_{[t]}^{\text{intra}}} \in \mathbb{R}^{C\times d}$$

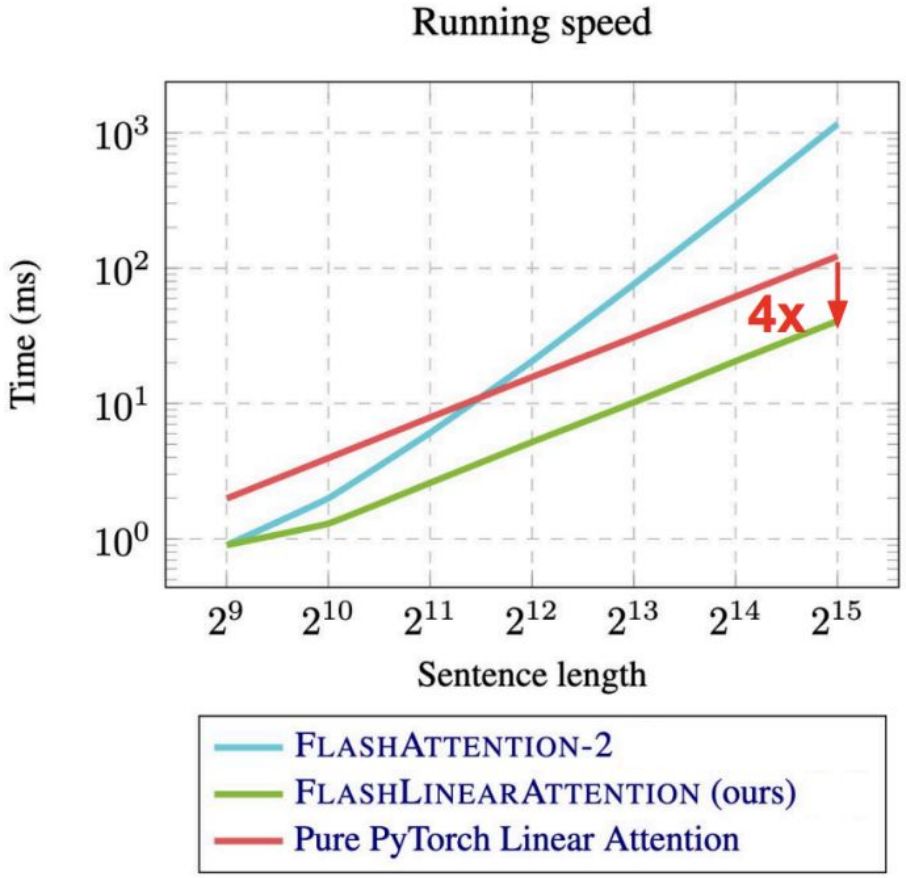
Computational Complexity: $\mathcal{O}(C^2d + Cd^2)$ per chunk.
$\mathcal{O}(Ld^2 + LCd)$ for the entire sequence.

Chunkwise Parallel Form

- **Total complexity**: $\mathcal{O}(Ld^2 + LdC)$, achieving **subquadratic complexity** in sequence length when C is small.
- **Practical settings**: C is typically set to $\{64, 128, 256\}$.
- **Extensibility**: Can be generalized to **linear attention with decay and delta rule** (to be discussed later).
- **Adoption**: The de facto standard for training modern linear attention models, including:
  - **Mamba2**, **Based**, **GLA**, **DeltaNet**, **Lightning Attention**, **mLSTM**, and others.

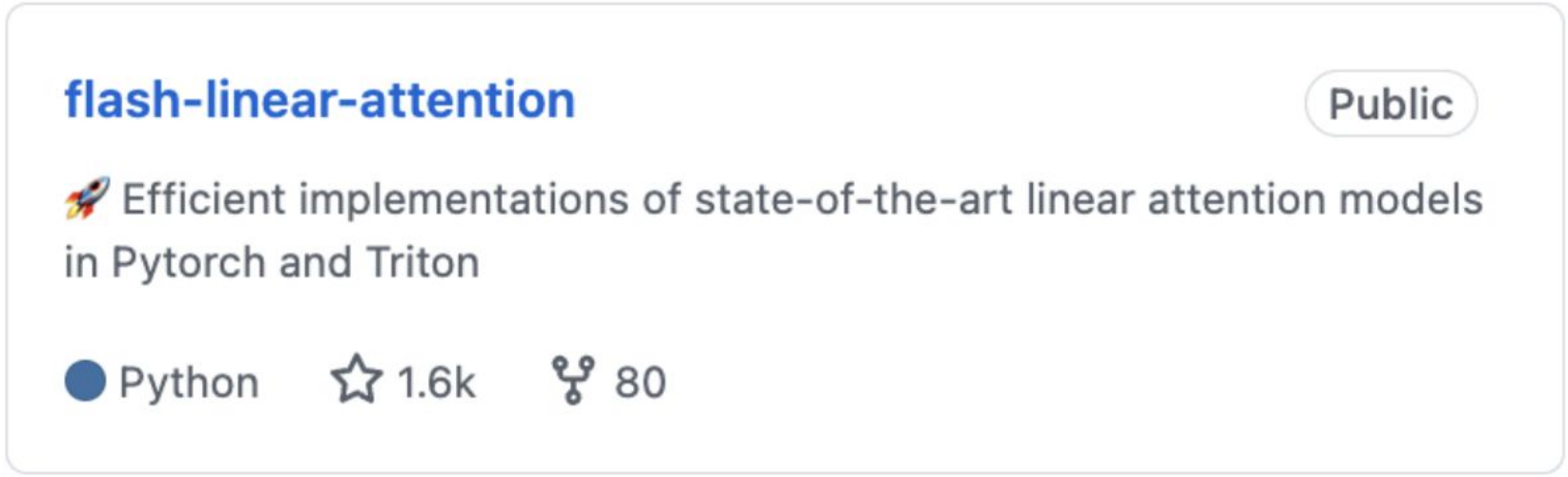
Flash Linear Attention





I/O optimization significantly improves the wall-clock time.

Flash Linear Attention



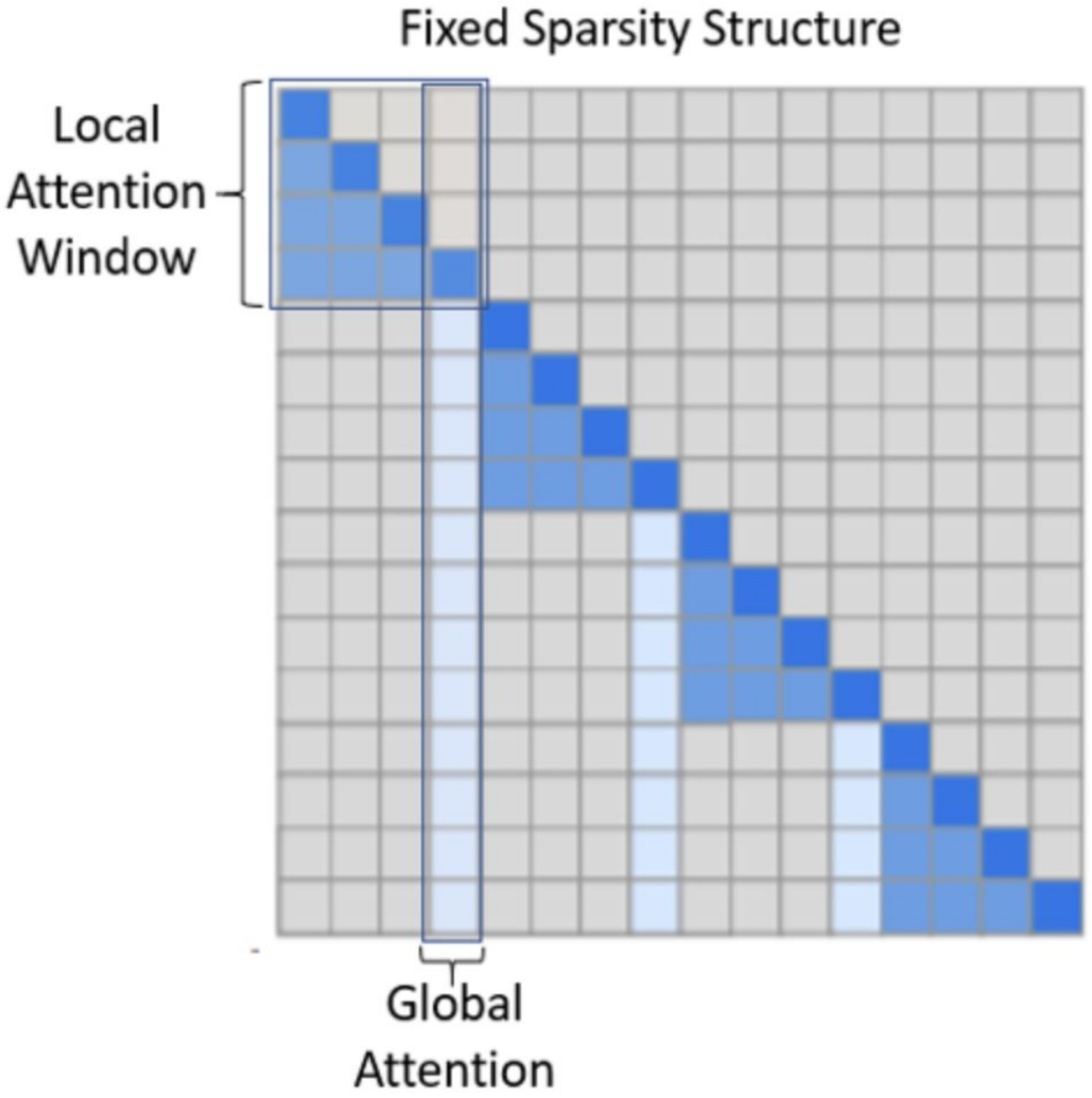
The Flash Linear Attention library provides hardware-efficient implementation of various linear attention models.

- RetNet, GLA, Based, HGRN2, RWKV6, GSA, Mamba2, DeltaNet, Gated DeltaNet, RWKV7 ...

Summary: Linear Attention

- Linear attention = Softmax attention - softmax.
- Linear attention = Linear RNN + matrix-valued hidden state.
- The chunkwise parallel form is more hardware-friendly than the recurrent and parallel forms.
- Flash Linear Attention is an I/O-aware implementation of the chunkwise parallel form.

Sparse Attention



Sliding Window Attention

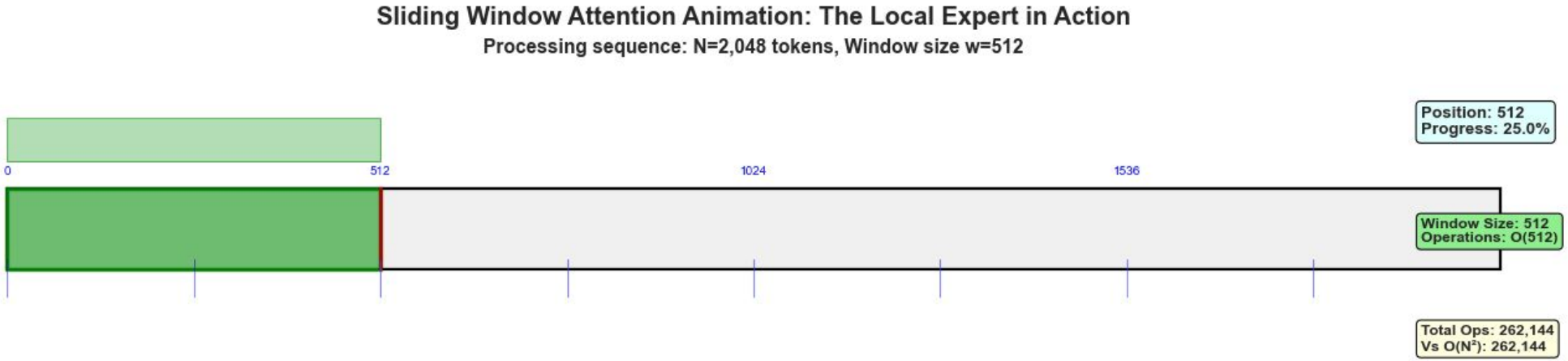
The Goal: To perfectly preserve the fine-grained information in the most immediate local context.

The Mechanism: This is the most straightforward branch. It isolates the most recent w=512 key-value pairs and performs a standard attention calculation on them

The Math: The calculation is a standard attention formula, applied only to the local window:

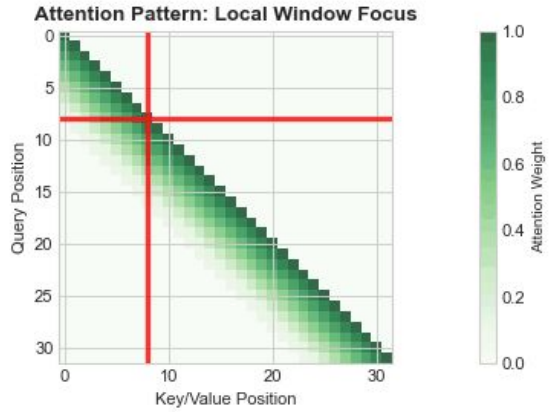
$$\text{Output}_{\text{win}} = \text{Attention}(q_t, K_{t-w:t}, V_{t-w:t})$$

Sliding Window Attention

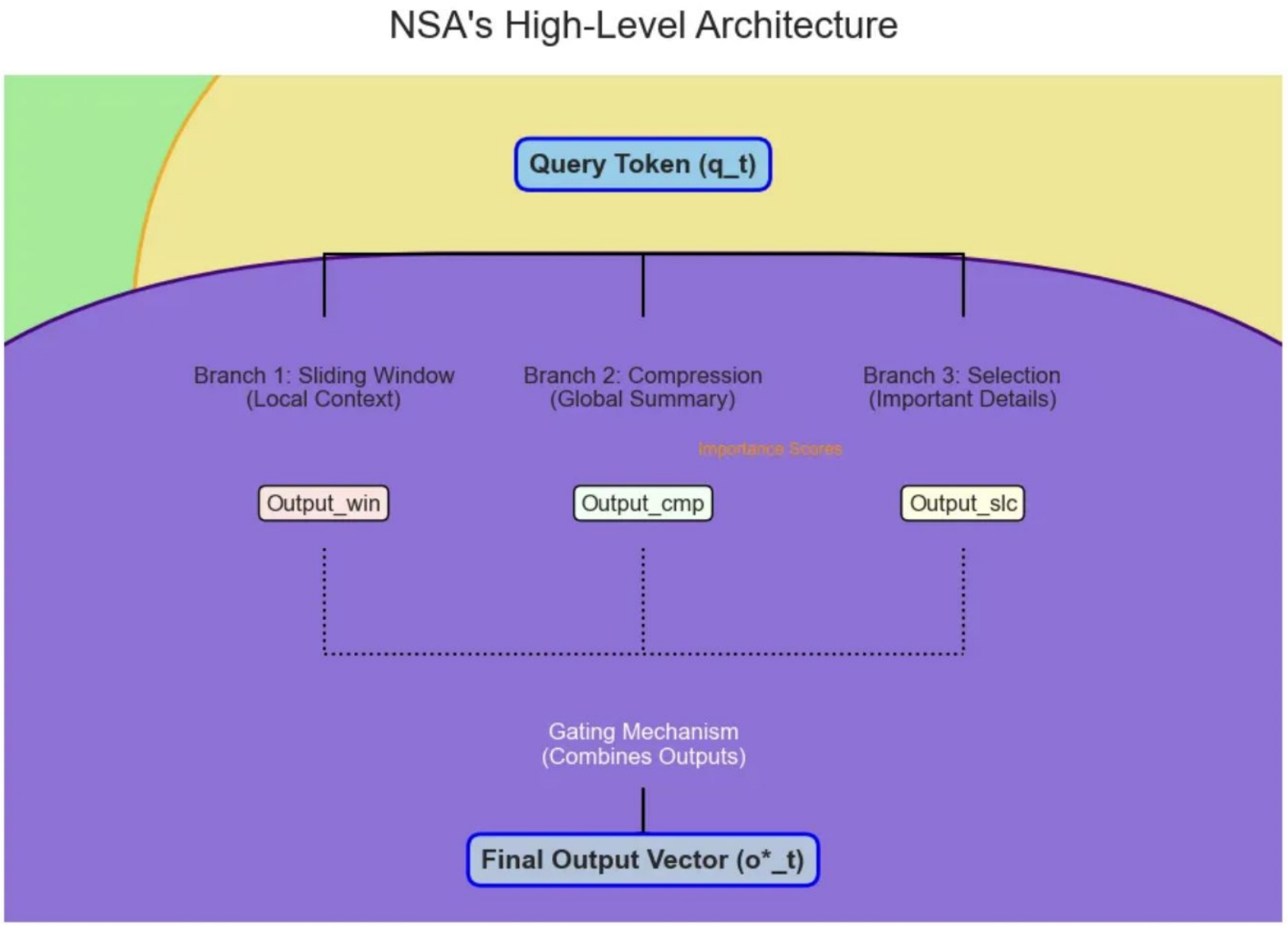


$$\text{Output}_t = \text{Attention}(q_t, K_{max(0,t-w):t}, V_{max(0,t-w):t})$$

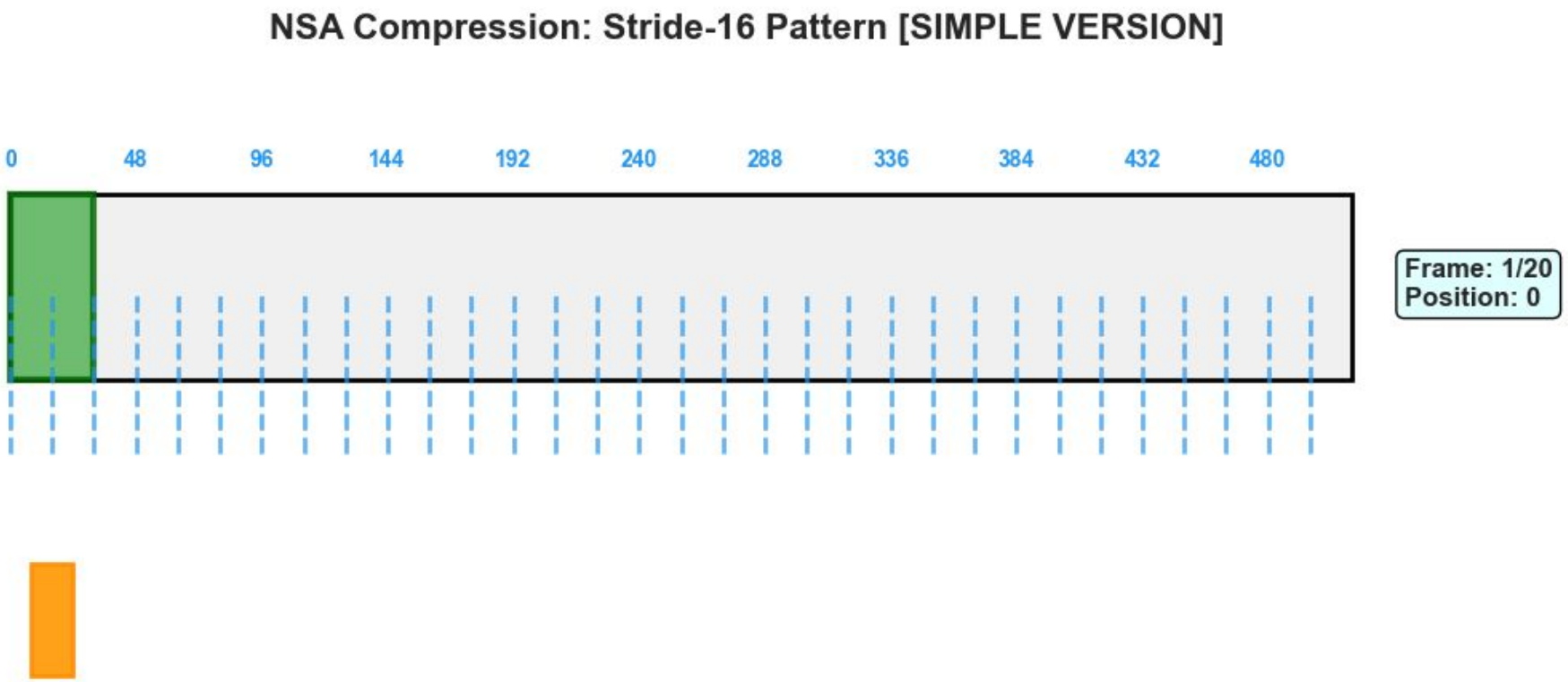
LOCAL EXPERT BENEFITS:
- Constant O(512) operations per token
- Perfect fine-grained pattern capture
- Linear O(N×512) total complexity
- Excellent cache locality



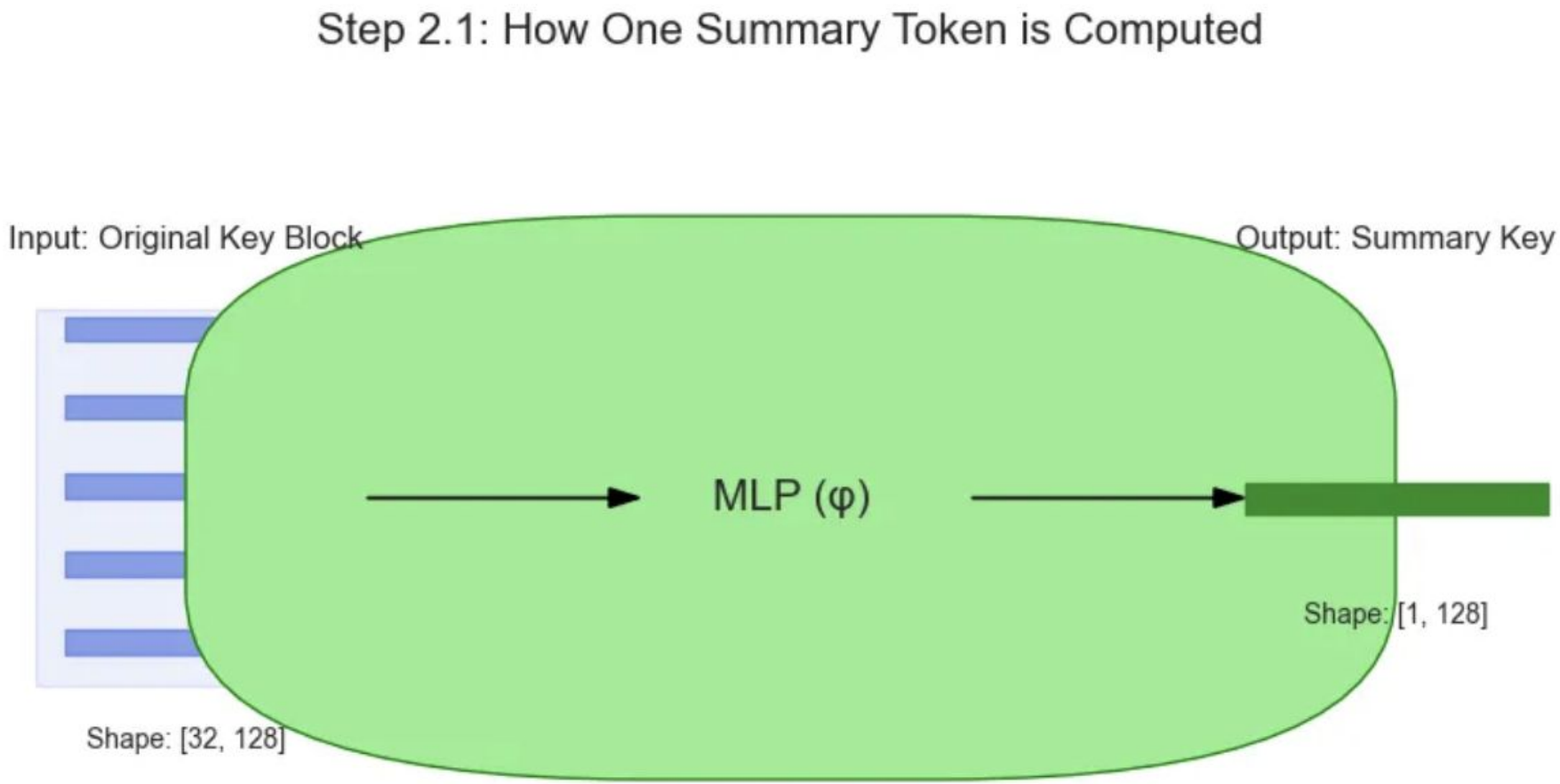
Native Sparse Attention (NSA)



NSA Compression



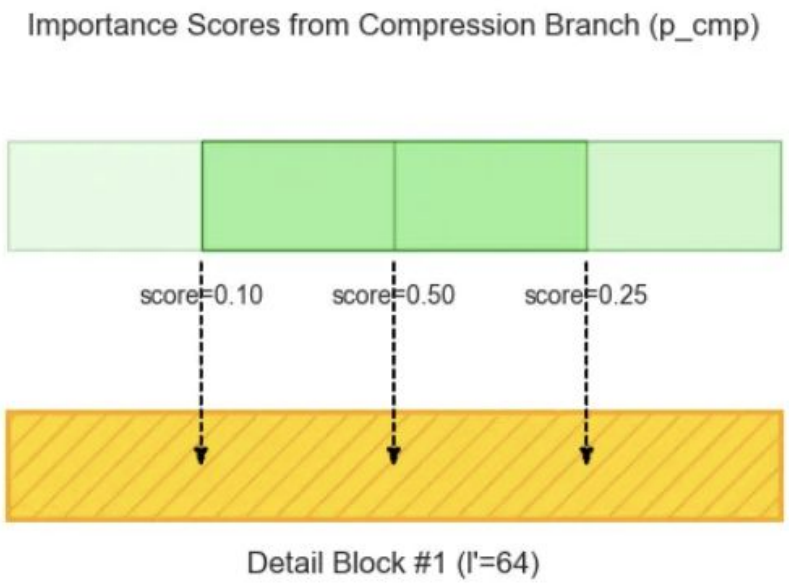
NSA Compression



This diagram illustrates a single block of 32 key vectors being processed by the MLP to produce one summary key vector. Similarly for value vectors as well

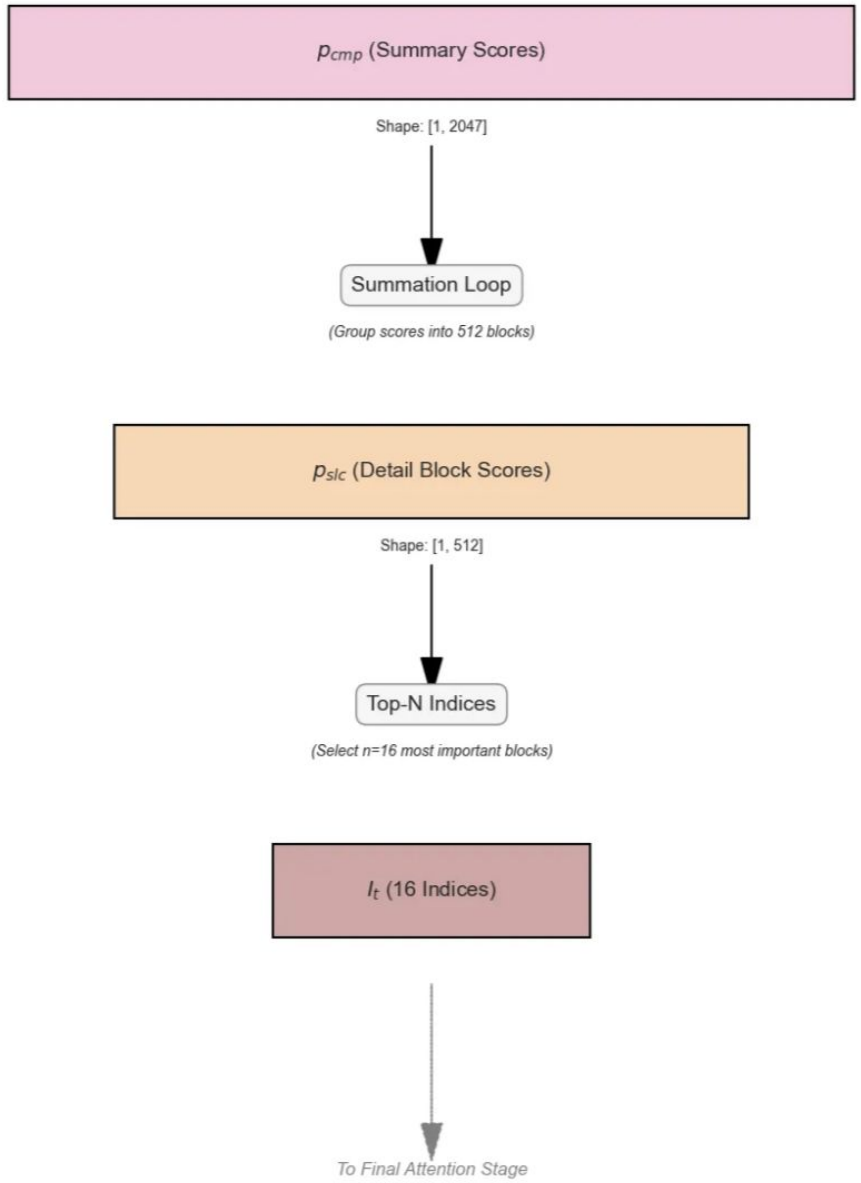
Token Selection

Goal: To capture critical, long-range, fine-grained dependencies by focusing compute only on the most relevant parts of the text.

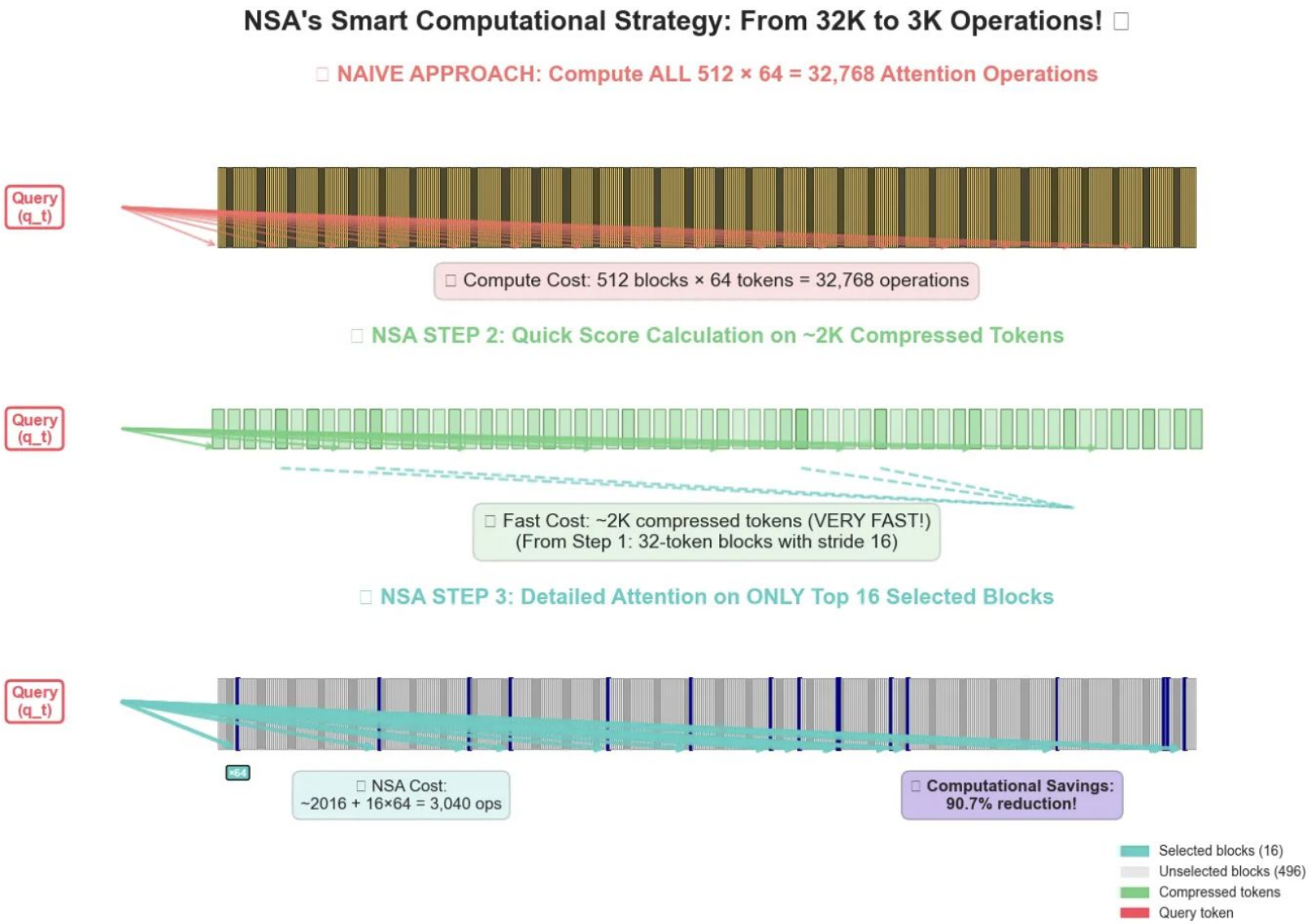


Importance for Detail Block #1 =
Score(Sum. #1) + Score(Sum. #2) + Score(Sum. #3)
= 0.1 + 0.5 + 0.25 = 0.85

Token Selection



NSA Results


NSA's Smart Computational Strategy: From 32K to 3K Operations!
NAIVE APPROACH: Compute ALL 512 × 64 = 32,768 Attention Operations
Query (q_t)
Compute Cost: 512 blocks × 64 tokens = 32,768 operations
NSA STEP 2: Quick Score Calculation on ~2K Compressed Tokens
Query (q_t)
Fast Cost: ~2K compressed tokens (VERY FAST!)
(From Step 1: 32-token blocks with stride 16)
NSA STEP 3: Detailed Attention on ONLY Top 16 Selected Blocks
Query (q_t)
×64
NSA Cost:
~2016 + 16×64 = 3,040 ops
Computational Savings:
90.7% reduction!
Selected blocks (16)
Unselected blocks (496)
Compressed tokens
Query token

NSA Results

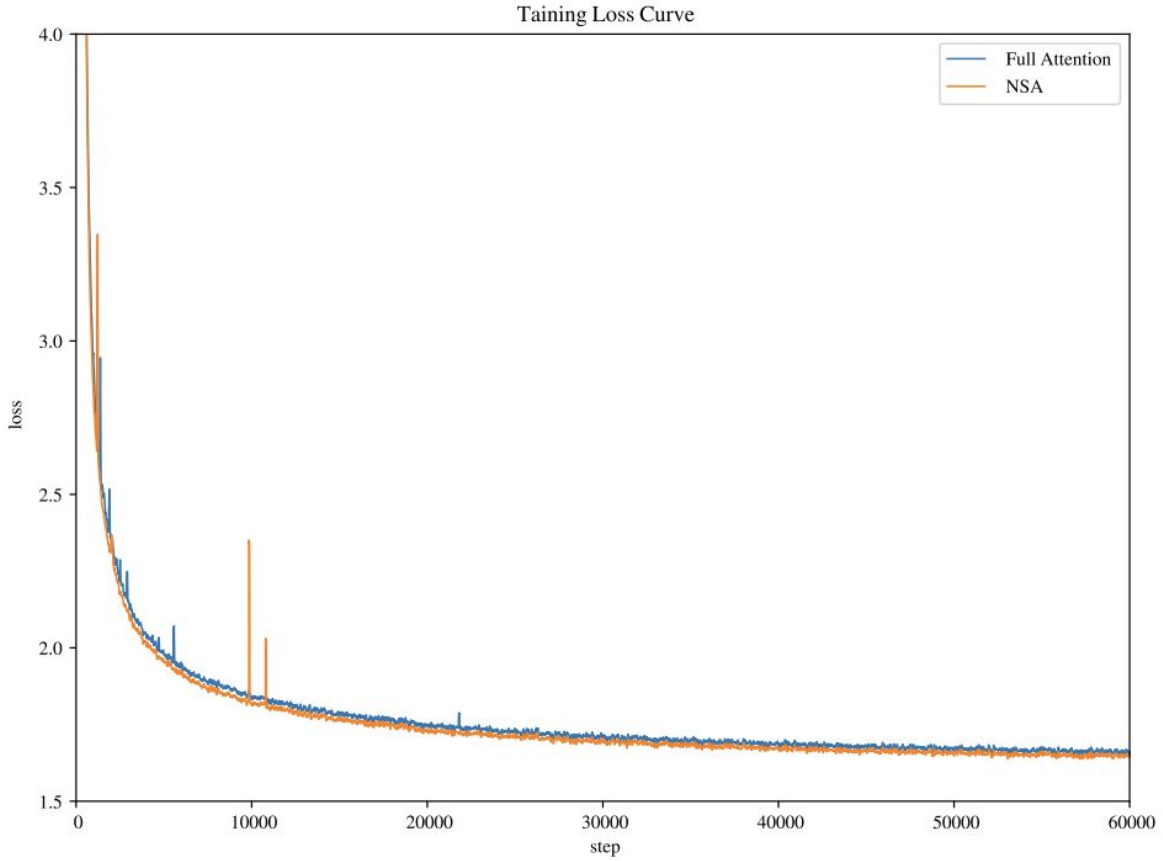


Figure 4 | Pretraining loss comparison between Full Attention and our NSA on 27B-parameter model. Both models exhibit stable convergence, with NSA achieving lower loss values.

Model	MMLU Acc. 5-shot	MMLU-PRO Acc. 5-shot	CMMLU Acc. 5-shot	BBH Acc. 3-shot	GSM8K Acc. 8-shot	MATH Acc. 4-shot	DROP F1 1-shot	MBPP Pass@1 3-shot	HumanEval Pass@1 0-shot	Avg.
Full Attn	**0.567**	0.279	0.576	0.497	0.486	0.263	0.503	**0.482**	0.335	0.443
NSA	0.565	**0.286**	**0.587**	**0.521**	**0.520**	**0.264**	**0.545**	0.466	**0.348**	**0.456**

Thanks

Q&A

Prof. Yu Cheng
chengyu@cse.cuhk.edu.hk